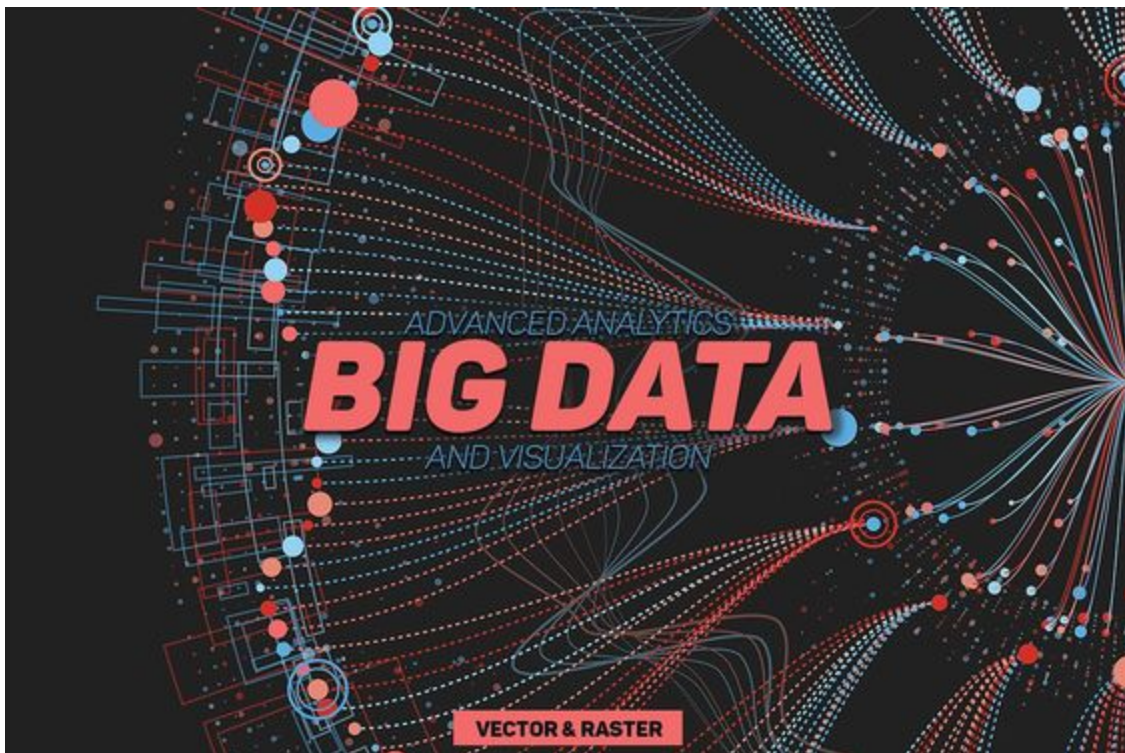




cse 2122 (lab)

🕒 Created	@February 7, 2023 6:28 PM
🏷️ Tags	



Lab Course Contents/List of Experiments:

1. Design, develop and implement a program for the following Array operations
 - a. Creating an Array of N Integer Elements
 - b. Display of Array Elements with Suitable Headings
 - c. Inserting an Element (ELEM) at a given valid Position (POS)
 - d. Deleting an Element at a given valid Position (POS)

▼ Answer

```

#include <stdio.h>

int main() {
    int arr[100], n, pos, elem, i;

    // Creating an Array of N Integer Elements
    printf("Enter the size of the array: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Display of Array Elements with Suitable Headings
    printf("\nArray Elements:\n");
    for (i = 0; i < n; i++) {
        printf("Element %d: %d\n", i+1, arr[i]);
    }

    // Inserting an Element (ELEM) at a given valid Position
    printf("\nEnter the element you want to insert: ");
    scanf("%d", &elem);
    printf("Enter the position where you want to insert the element: ");
    scanf("%d", &pos);
    if (pos <= n && pos > 0) {
        for (i = n-1; i >= pos-1; i--) {
            arr[i+1] = arr[i];
        }
        arr[pos-1] = elem;
        n++;
        printf("\nElement inserted successfully!\n");
    } else {
        printf("\nInvalid position entered. Please try again\n");
    }
}

```

```

// Deleting an Element at a given valid Position (POS)
printf("\nEnter the position of the element you want to delete: ");
scanf("%d", &pos);
if (pos <= n && pos > 0) {
    for (i = pos-1; i < n-1; i++) {
        arr[i] = arr[i+1];
    }
    n--;
    printf("\nElement deleted successfully!\n");
} else {
    printf("\nInvalid position entered. Please try again\n");
}

// Display of Array Elements after Insertion and Deletion
printf("\nArray Elements:\n");
for (i = 0; i < n; i++) {
    printf("Element %d: %d\n", i+1, arr[i]);
}

return 0;
}

```

2. Design, Develop and Implement a program for the following Array operations
 - a. Creating an Array of N Integer Elements
 - b. Sort the elements using Bubble Sort Algorithm
 - c. Search an item using Linear Search Algorithm
 - d. Search an item using Binary Search Algorithm

▼ Answer

```

#include <stdio.h>

// Function to create an array of N integer elements
void createArray(int arr[], int n) {
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {

```

```

        scanf("%d", &arr[i]);
    }
}

// Function to perform bubble sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    printf("Sorted array: ");
    displayArray(arr, n);
}

// Function to perform linear search
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

// Function to perform binary search
int binarySearch(int arr[], int n, int key) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (arr[mid] == key) {

```

```

        return mid;
    } else if (arr[mid] < key) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}
return -1;
}

// Function to display the elements of an array
void displayArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int n, arr[100], key;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    createArray(arr, n);
    bubbleSort(arr, n);
    printf("Enter the element to search: ");
    scanf("%d", &key);
    int index = linearSearch(arr, n, key);
    if (index != -1) {
        printf("%d found at index %d\n", key, index);
    } else {
        printf("%d not found in the array\n", key);
    }
    index = binarySearch(arr, n, key);
    if (index != -1) {
        printf("%d found at index %d\n", key, index);
    } else {

```

```

        printf("%d not found in the array\n", key);
    }
    return 0;
}

```

3. Design, Develop and Implement a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX)
- Push an Element on to Stack
 - Pop an Element from Stack
 - Display the status of Stack

▼ Answer

```

#include <stdio.h>
#define MAX 100 // Define maximum size of stack

int stack[MAX]; // Declare integer array stack to hold elements
int top = -1; // Set initial value of top element to -1

void push(int element) { // Define function for pushing an element
    if (top == MAX - 1) { // Check if the stack is full
        printf("Stack Overflow\n"); // Print error message if full
    } else {
        top++; // Increment top to point to the next empty location
        stack[top] = element; // Add element to the top of the stack
        printf("%d pushed to stack\n", element); // Print success message
    }
}

void pop() { // Define function for popping an element from the stack
    if (top == -1) { // Check if the stack is empty
        printf("Stack Underflow\n"); // Print error message if empty
    } else {
        printf("%d popped from stack\n", stack[top]); // Print the popped element
        top--; // Decrement top to remove the element
    }
}

```

```

    }
}

void display() { // Define function to display the elements :
    if (top == -1) { // Check if the stack is empty
        printf("Stack is empty\n"); // Print message indicating
    } else {
        printf("Elements in stack are:\n"); // Print a message
        for (int i = top; i >= 0; i--) { // Loop through the
            printf("%d\n", stack[i]); // Print the element at
        }
    }
}

int main() {
    int choice, element; // Declare variables for user input
    do { // Repeat until the user chooses to exit
        printf("\nMenu:\n1. Push Element\n2. Pop Element\n3.
        scanf("%d", &choice); // Get user's choice
        switch (choice) {
            case 1:
                printf("Enter the element to be pushed: ");
                scanf("%d", &element); // Get the element to
                push(element); // Call the push function with
                break;
            case 2:
                pop(); // Call the pop function
                break;
            case 3:
                display(); // Call the display function
                break;
            case 4:
                printf("Exiting..."); // Print a message indicating
                break;
            default:
                printf("Invalid Choice\n"); // Print an error

```

```

    }
    } while (choice != 4); // Keep looping until the user chooses
    return 0; // Terminate the program
}

```

4. Design, Develop and Implement a Program for converting an Infix Expression to Postfix Expression.

Program should support for both parenthesized and free parenthesized expressions with the operators:

+, -, *, /, % (Remainder), ^ (Power) and alphanumeric operands.

▼ Answer

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

// Define stack structure
struct Stack {
    int top;
    char items[MAX];
};

// Declare stack functions
void push(struct Stack *s, char item);
char pop(struct Stack *s);
int precedence(char symbol);

int main() {
    struct Stack s;
    s.top = -1;
    char infix[MAX], postfix[MAX], item;
    int i = 0, j = 0;
}

```

```

// Input infix expression from user
printf("Enter Infix Expression: ");
fgets(infix, sizeof(infix), stdin);

// Remove newline character at the end of input string
if (infix[strlen(infix) - 1] == '\n') {
    infix[strlen(infix) - 1] = '\0';
}

// Process each symbol in the infix expression
while (infix[i] != '\0') {
    item = infix[i];

    // If symbol is alphanumeric, add it to the postfix expression
    if (isalnum(item)) {
        postfix[j] = item;
        j++;
    }

    // If symbol is an opening parenthesis, push it onto the stack
    else if (item == '(') {
        push(&s, item);
    }

    // If symbol is a closing parenthesis, pop all symbols from the stack until the opening parenthesis is found
    else if (item == ')') {
        while (s.items[s.top] != '(') {
            postfix[j] = pop(&s);
            j++;
        }
        pop(&s); // Pop the opening parenthesis from the stack
    }

    // If symbol is an operator, pop all operators with lower precedence
    else {

```

```

        while (s.top != -1 && precedence(s.items[s.top])
               postfix[j] = pop(&s);
               j++;
        }
        push(&s, item); // Push the current operator onto the stack
    }

    i++; // Move to the next symbol in the infix expression
}

// Pop all remaining symbols from the stack and add them to the postfix expression
while (s.top != -1) {
    postfix[j] = pop(&s);
    j++;
}

postfix[j] = '\0'; // Add null character at the end of the postfix expression

// Output the postfix expression
printf("Postfix Expression: %s\n", postfix);

return 0;
}

// Function to push an item onto the stack
void push(struct Stack *s, char item) {
    if (s->top == MAX - 1) { // Check if stack is full
        printf("Stack Overflow\n");
        exit(1); // Terminate program with error code 1
    } else {
        s->top++; // Increment top pointer to point to next free space
        s->items[s->top] = item; // Add item to stack
    }
}

// Function to pop an item from the stack

```

```

char pop(struct Stack *s) {
    char item;
    if (s->top == -1) { // Check if stack is empty
        printf("Stack Underflow\n");
        exit(1); // Terminate program with error code 1
    } else {
        item = s->items[s->top]; // Get top item from stack
        s->top--; // Decrement top pointer to remove item from stack
        return item; // Return popped item
    }
}

// Function to get the precedence of an operator
int precedence(char symbol) {
    switch(symbol) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
        case '%':
            return 2;
        case '^':
            return 3;
        default:
            return 0; // Return 0 for alphanumeric operands
    }
}

```

5. Design, Develop and Implement a Program for the following Stack Applications
 - a. Evaluation of Postfix expression with single digit operands and operators: +, -, *, /, %, ^
 - b. Solving Tower of Hanoi problem with n disks

▼ Answer

a.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

// Define stack structure
struct Stack {
    int top;
    int items[MAX];
};

// Declare stack functions
void push(struct Stack *s, int item);
int pop(struct Stack *s);

int main() {
    struct Stack s;
    s.top = -1;
    char postfix[MAX], item;
    int i = 0, operand1, operand2, result;

    // Input postfix expression from user
    printf("Enter Postfix Expression: ");
    fgets(postfix, sizeof(postfix), stdin);

    // Remove newline character at the end of input string
    if (postfix[strlen(postfix) - 1] == '\n') {
        postfix[strlen(postfix) - 1] = '\0';
    }

    // Process each symbol in the postfix expression
    while (postfix[i] != '\0') {
        item = postfix[i];

```

```

// If symbol is an operand, push it onto the stack
if (isdigit(item)) {
    push(&s, item - '0'); // Convert character digit to integer
}

// If symbol is an operator, pop two operands from the stack
else {
    operand2 = pop(&s);
    operand1 = pop(&s);

    switch (item) {
        case '+':
            result = operand1 + operand2;
            break;
        case '-':
            result = operand1 - operand2;
            break;
        case '*':
            result = operand1 * operand2;
            break;
        case '/':
            result = operand1 / operand2;
            break;
        case '%':
            result = operand1 % operand2;
            break;
        case '^':
            result = 1;
            for (int j = 1; j <= operand2; j++) {
                result *= operand1;
            }
            break;
        default:
            printf("Invalid Operator\n");
            exit(1); // Terminate program with error
    }
}

```

```

    }

    push(&s, result); // Push the result of the op
}

i++; // Move to the next symbol in the postfix exp
}

result = pop(&s); // Pop the final result from the stack

if (s.top != -1) { // Check if there are extra operands
    printf("Invalid Expression\n");
    exit(1); // Terminate program with error code 1
}

// Output the result of the evaluation
printf("Result: %d\n", result);

return 0;
}

// Function to push an item onto the stack
void push(struct Stack *s, int item) {
    if (s->top == MAX - 1) { // Check if stack is full
        printf("Stack Overflow\n");
        exit(1); // Terminate program with error code 1
    } else {
        s->top++; // Increment top pointer to point to next
        s->items[s->top] = item; // Add item to stack
    }
}

// Function to pop an item from the stack
int pop(struct Stack *s) {
    int item;
    if (s->top == -1) { // Check if stack is empty

```

```

        printf("Stack Underflow\n");
        exit(1); // Terminate program with error code 1
    } else {
        item = s->items[s->top]; // Get top item from stack
        s->top--; // Decrement top pointer to remove item
        return item; // Return popped item
    }
}

```

b.

```

#include <stdio.h>
#include <stdlib.h>

// Define stack structure
struct Stack {
    int top;
    int items[20];
};

// Declare stack functions
void push(struct Stack *s, int item);
int pop(struct Stack *s);

// Function to solve the Tower of Hanoi problem for n disks
void towerOfHanoi(int n) {
    struct Stack source, auxiliary, destination;
    int totalMoves = (1 << n) - 1; // Calculate total number of moves

    // Initialize stacks
    source.top = -1;
    auxiliary.top = -1;
    destination.top = -1;

    // Push all disks onto the source tower
    for (int i = n; i >= 1; i--) {

```

```

        push(&source, i);
    }

    // Alternate between the three towers to move all the
    for (int i = 1; i <= totalMoves; i++) {
        if (i % 3 == 1) { // Move disk from source to destination
            if (source.top == -1) {
                printf("Invalid Move\n");
                exit(1); // Terminate program with error code 1
            }
            else {
                push(&destination, pop(&source));
                printf("Move disk %d from source to destination\n", i);
            }
        }
        else if (i % 3 == 2) { // Move disk from source to auxiliary
            if (auxiliary.top == -1 || (source.top != -1 &
                push(&auxiliary, pop(&source));
                printf("Move disk %d from source to auxiliary\n", i);
            }
            else {
                push(&source, pop(&auxiliary));
                printf("Move disk %d from auxiliary to source\n", i);
            }
        }
        else { // Move disk from auxiliary to destination
            if (destination.top == -1 || (auxiliary.top != -1 &
                push(&destination, pop(&auxiliary));
                printf("Move disk %d from auxiliary to destination\n", i);
            }
            else {
                push(&auxiliary, pop(&destination));
                printf("Move disk %d from destination to auxiliary\n", i);
            }
        }
    }
}

```

```

}

// Function to push an item onto the stack
void push(struct Stack *s, int item) {
    if (s->top == 19) { // Check if stack is full
        printf("Stack Overflow\n");
        exit(1); // Terminate program with error code 1
    } else {
        s->top++; // Increment top pointer to point to next
        s->items[s->top] = item; // Add item to stack
    }
}

// Function to pop an item from the stack
int pop(struct Stack *s) {
    int item;
    if (s->top == -1) { // Check if stack is empty
        printf("Stack Underflow\n");
        exit(1); // Terminate program with error code 1
    } else {
        item = s->items[s->top]; // Get top item from stack
        s->top--; // Decrement top pointer to remove item
        return item; // Return popped item
    }
}

int main() {
    int n;
    printf("Enter the number of disks: ");
    scanf("%d", &n);
    towerOfHanoi(n);
    return 0;
}

```

6. Design, Develop and Implement a Program for the following Recursion Applications

- a. Calculate the factorial of n
- b. Display the Fibonacci sequence of n numbers

▼ Answer

```
#include <stdio.h>

// Function to calculate factorial using recursion
int factorial(int n)
{
    if (n == 0) // Base case
        return 1;
    else
        return n * factorial(n-1); // Recursive call
}

// Function to display Fibonacci sequence up to n numbers
void fibonacci(int n)
{
    int a = 0, b = 1, c, i;
    printf("Fibonacci sequence: ");
    for (i = 0; i < n; i++)
    {
        if (i <= 1)
            c = i;
        else
        {
            c = a + b;
            a = b;
            b = c;
        }
        printf("%d ", c);
    }
    printf("\n");
}
```

```

}

// Main function to test recursion applications
int main()
{
    int num;
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    // Calculate factorial
    printf("Factorial of %d is %d\n", num, factorial(num));

    // Display Fibonacci sequence
    fibonacci(num);

    return 0;
}

```

7. Design, Develop and Implement a menu driven Program for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX)
- Insert an Element on to Circular QUEUE
 - Delete an Element from Circular QUEUE
 - Display the status of Circular QUEUE

▼ Answer

```

#include <stdio.h>
#define MAX 5

char queue[MAX];
int rear = -1;
int front = -1;

// Function to insert an element into the circular queue
void enqueue(char element)

```

```

{
    if ((front == 0 && rear == MAX-1) || (front == rear+1)
    {
        printf("Queue Overflow!\n");
        return;
    }
    else if (front == -1 && rear == -1)
    {
        front = 0;
        rear = 0;
    }
    else if (rear == MAX-1)
        rear = 0;
    else
        rear++;

    queue[rear] = element;
}

```

// Function to delete an element from the circular queue
void dequeue()

```

{
    if (front == -1 && rear == -1)
    {
        printf("Queue Underflow!\n");
        return;
    }
    else if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else if (front == MAX-1)
        front = 0;
    else
        front++;
}

```

```

}

// Function to display the status of the circular queue
void display()
{
    int i;
    if (front == -1 && rear == -1)
    {
        printf("Queue is empty!\n");
        return;
    }
    printf("Circular Queue: ");
    if (rear >= front)
    {
        for (i = front; i <= rear; i++)
            printf("%c ", queue[i]);
    }
    else
    {
        for (i = front; i < MAX; i++)
            printf("%c ", queue[i]);
        for (i = 0; i <= rear; i++)
            printf("%c ", queue[i]);
    }
    printf("\n");
}

// Main function to test circular queue operations
int main()
{
    int choice;
    char value;

    do
    {

```

```

printf("\n\n*** Circular Queue Operations ***\n");
printf("1. Insert Element\n2. Delete Element\n3. I
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        printf("Enter the element to be inserted:");
        scanf(" %c", &value);
        enqueue(value);
        break;
    case 2:
        dequeue();
        break;
    case 3:
        display();
        break;
    case 4:
        printf("Exiting Program!\n");
        break;
    default:
        printf("Invalid Choice! Try Again.\n");
}
} while (choice != 4);

return 0;
}

```

8. **Design, Develop and Implement a program for the following operations on Singly Linked List (SLL) of Student Data with the fields: ID, Name, Dept, Sem, Mobile**
 - a. Create a SLL of N Students Data by using front insertion.
 - b. Display the status of SLL and count the number of nodes in it
 - c. Perform Insertion / Deletion at End of SLL

d. Perform Insertion / Deletion at Front of SLL

e. Perform Insertion / Deletion after a given node

▼ Answer

9. Design, Develop and Implement a program for the following operations on Binary Search Tree (BST) of Integers
- Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2
 - Traverse the BST in Inorder, Preorder and Post Order
 - Search the BST for a given element (KEY) and report the appropriate message

▼ Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node* left;
    struct Node* right;
};

typedef struct Node node;

node* createNode(int data)
{
    node* new_node = (node*) malloc(sizeof(node));
    if (new_node == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(0);
    }

    new_node->data = data;
    new_node->left = NULL;
```

```

        new_node->right = NULL;

        return new_node;
    }

node* insert(node* root, int data)
{
    if (root == NULL)
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}

void inorderTraversal(node* root)
{
    if (root != NULL)
    {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}

void preorderTraversal(node* root)
{
    if (root != NULL)
    {
        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}

```

```

}

void postorderTraversal(node* root)
{
    if (root != NULL)
    {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%d ", root->data);
    }
}

node* search(node* root, int key)
{
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

int main()
{
    node* root = NULL;
    int n, data, key;

    printf("Enter the number of integers to insert in BST\n");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++)
    {
        printf("Enter an integer: ");
        scanf("%d", &data);
        root = insert(root, data);
    }
}

```

```

    }

    printf("\nInorder Traversal of BST: ");
    inorderTraversal(root);

    printf("\nPreorder Traversal of BST: ");
    preorderTraversal(root);

    printf("\nPostorder Traversal of BST: ");
    postorderTraversal(root);

    printf("\nEnter an integer to search in BST: ");
    scanf("%d", &key);

    if (search(root, key) != NULL)
        printf("%d is found in BST.\n", key);
    else
        printf("%d is not found in BST.\n", key);

    return 0;
}

```

10. Design, Develop and Implement a program for the following operations on Complete Binary Search Tree of Integers
- Create a Heap of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2
 - Sort the list using Heap Sort Algorithm.

▼ Answer

```

#include <stdio.h>
#include <stdlib.h>

void swap(int* a, int* b)
{
    int temp = *a;
    *a = *b;
}

```

```

        *b = temp;
    }

void heapify(int arr[], int n, int i)
{
    int largest = i; // Initialize largest as root
    int l = 2 * i + 1; // left = 2*i + 1
    int r = 2 * i + 2; // right = 2*i + 2

    // If left child is larger than root
    if (l < n && arr[l] > arr[largest])
        largest = l;

    // If right child is larger than largest so far
    if (r < n && arr[r] > arr[largest])
        largest = r;

    // If largest is not root
    if (largest != i)
    {
        swap(&arr[i], &arr[largest]);

        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // One by one extract an element from heap
    for (int i = n - 1; i >= 0; i--)
    {

```

```

        // Move current root to end
        swap(&arr[0], &arr[i]);

        // call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}

int main()
{
    int n, arr[100];

    printf("Enter the number of integers to sort: ");
    scanf("%d", &n);

    printf("Enter %d integers: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("\nSorted array: ");
    printArray(arr, n);

    return 0;
}

```

11. Design, Develop and Implement a Program for the following operations on Graph(G) of Cities

- a. Create a Graph of N cities using Adjacency Matrix.
- b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method

▼ Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 50

int adj_matrix[MAX_SIZE][MAX_SIZE];
int visited[MAX_SIZE];
int n;

void createGraph()
{
    int max_edges, origin, dest;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    max_edges = n * (n - 1);

    for (int i = 0; i < max_edges; i++)
    {
        printf("Enter edge %d (or -1 -1 to quit): ", i + 1);
        scanf("%d %d", &origin, &dest);

        if (origin == -1 && dest == -1)
            break;
        else if (origin >= n || dest >= n || origin < 0 || dest < 0)
        {
            printf("Invalid edge!\n");
            i--;
        }
    }
}
```

```

        else
        {
            adj_matrix[origin][dest] = 1;
        }
    }
}

void printGraph()
{
    printf("\nAdjacency Matrix:\n");
    printf("-----\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            printf("%d ", adj_matrix[i][j]);
        printf("\n");
    }
}

void dfs(int v)
{
    visited[v] = 1;
    printf("%d ", v);

    for (int i = 0; i < n; i++)
    {
        if (adj_matrix[v][i] && !visited[i])
            dfs(i);
    }
}

void bfs(int v)
{
    int queue[MAX_SIZE], front = -1, rear = -1;
    queue[++rear] = v;

```

```

    visited[v] = 1;

    while (front != rear)
    {
        v = queue[++front];
        printf("%d ", v);

        for (int i = 0; i < n; i++)
        {
            if (adj_matrix[v][i] && !visited[i])
            {
                visited[i] = 1;
                queue[++rear] = i;
            }
        }
    }
}

int main()
{
    int start;

    createGraph();
    printGraph();

    printf("\nEnter the starting node: ");
    scanf("%d", &start);

    printf("\nDFS traversal from node %d: ", start);
    dfs(start);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("\nBFS traversal from node %d: ", start);
    bfs(start);
}

```

```

        return 0;
    }

```

12. Design, Develop and Implement a Program for the following operations on Graph(G) of Cities
- Create a Weighted Graph of N cities using Adjacency Matrix.
 - Print the shortest path from a weighted graph using Warshall's Algorithm.

▼ Answer

```

#include <stdio.h>
#include <stdlib.h>

#define INF 99999
#define MAX_SIZE 50

int adj_matrix[MAX_SIZE][MAX_SIZE];
int n;

void createGraph()
{
    int max_edges, origin, dest, weight;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    max_edges = n * (n - 1);

    for (int i = 0; i < max_edges; i++)
    {
        printf("Enter edge %d (or -1 -1 to quit): ", i + 1);
        scanf("%d %d", &origin, &dest);

        if (origin == -1 && dest == -1)
            break;
    }
}

```

```

        else if (origin >= n || dest >= n || origin < 0 |
        {
            printf("Invalid edge!\n");
            i--;
            continue;
        }

        printf("Enter weight: ");
        scanf("%d", &weight);

        adj_matrix[origin][dest] = weight;
    }
}

void printGraph()
{
    printf("\nAdjacency Matrix:\n");
    printf("-----\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (adj_matrix[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", adj_matrix[i][j]);
        }
        printf("\n");
    }
}

void warshall()
{
    int dist[n][n];

```

```

// Initialize the distance matrix with the adjacency matrix
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        dist[i][j] = adj_matrix[i][j];

// Calculate all pairs shortest path by comparing the
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];

printf("Shortest Path Matrix:\n");
printf("-----\n");

for(int i=0;i<n;i++){
    for(int j=0;j<n;j++)
        printf("%d\t",dist[i][j]);
    printf("\n");
}

}

int main()
{
    createGraph();
    printGraph();

    warshall();

    return 0;
}

```

13. Given a File of N employee records with a set K of Keys(4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of

memory addresses (2- digit) of locations in HT. Let the keys in K and addresses in L are Integers. Design and develop a Program in C that uses Hash function $H: K \rightarrow L$ as $H(K) = K \bmod m$ (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.

▼ Answer

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

int hash_table[MAX_SIZE];
int m;

void initHashTable()
{
    for (int i = 0; i < m; i++)
        hash_table[i] = -1;
}

int hashFunction(int key)
{
    return key % m;
}

void insert(int key, int value)
{
    int index = hashFunction(key);

    if (hash_table[index] == -1)
    {
        hash_table[index] = value;
        printf("Record with key %d inserted at index %d\n", key, index);
    }
}
```

```

else
{
    int i = (index + 1) % m;

    while (i != index)
    {
        if (hash_table[i] == -1)
        {
            hash_table[i] = value;
            printf("Record with key %d inserted at index %d\n", key, i);
            break;
        }

        i = (i + 1) % m;
    }

    if (i == index)
        printf("Hash table full!\n");
}
}

void display()
{
    printf("\nHash Table:\n");
    printf("-----\n");

    for (int i = 0; i < m; i++)
    {
        if (hash_table[i] == -1)
            printf("%d : Empty\n", i);
        else
            printf("%d : %d\n", i, hash_table[i]);
    }
}

int main()

```

```

{
    int n, key, value;

    printf("Enter number of employee records: ");
    scanf("%d", &n);

    printf("Enter size of Hash Table: ");
    scanf("%d", &m);

    initHashTable();

    for (int i = 0; i < n; i++)
    {
        printf("Enter employee record %d (key value): ", i);
        scanf("%d", &key);

        insert(key, i);
    }

    display();

    return 0;
}

```